

DSAA3073: Theories in Data Science

Week 5: Cross-Validation & Model Selection Criteria

Concise Learning Sheet

Wei Wang · DSA, HKUST(GZ) · 2026-06-27

Explorative projection

Opening Challenge

The Hyperparameter Selection Dilemma

You've trained a ridge regression model on 100 samples and need to select the regularization parameter λ . You have three approaches:

1. **Validation set:** Split data 70/30, train on 70, validate on 30
2. **Leave-one-out CV:** Train 100 times, each time leaving out one sample
3. **AIC:** Use information criterion based on likelihood

Questions:

- Which method gives the most reliable estimate of test error?
- Which is computationally cheapest?
- Can you compute LOOCV without training 100 separate models?
- When would you prefer AIC over cross-validation?

This week formalizes these tradeoffs and reveals surprising computational shortcuts for linear models.

Introduction

Here's the central challenge of practical machine learning: after defining our **hypothesis class** (the family of models we consider, such as all linear functions or all degree-3 polynomials—introduced in Week 2), we still need to choose **hyperparameters** (configuration choices that control model complexity, such as regularization strength λ , rather than parameters learned from data like regression coefficients). We can't use the test set—that would contaminate our evaluation. We can't use training error—it's biased downward. **Cross-validation** (a resampling method that simulates fresh test sets from training data) solves this by partitioning training data into subsets for validation.

Last week we studied regularization as a solution to overfitting. But how do we choose the regularization parameter λ ? More generally, how do we select among competing models without using our test set?

Learning objectives: By the end of this week, you will:

1. Derive efficient LOOCV formulas for linear models
2. Understand the bias-variance tradeoff in k-fold CV
3. Derive AIC and BIC from information-theoretic principles
4. Compare consistency vs. prediction optimality of selection criteria
5. Apply structural risk minimization framework

← From Week 4: Bias-Variance & Regularization

Week 4 showed that model complexity (controlled by λ) trades off bias and variance. Cross-validation is how we empirically find the sweet spot—estimate test error for each λ and pick the minimizer.

→ Week 11 Preview: Deep Learning & Early Stopping

Week 11 shows how validation sets guide neural network training through early stopping—monitoring validation loss to decide when to stop training. The principles from this week underpin modern deep learning workflows.

Cross-Validation Framework

The core idea: partition your training data, use part to train and part to validate. Repeat multiple times to reduce variance. This estimates how well your training procedure generalizes to unseen data.

The Validation Principle

Why do we need cross-validation at all? Because **training error lies**. A complex model can perfectly fit training data (zero training error) while completely failing on test data. We need an honest estimate of test error without touching our actual test set.

Definition: Test Error Estimation

Given training data $S = \{(x_i, y_i)\}_{i=1}^n$, we want to estimate the **test error** (expected loss on new data from the same distribution) of the fitted model:

$$R_{\mathcal{D}}(\hat{f}) = \mathbb{E}_{x, y \sim \mathcal{D}} \left[(y - \hat{f}(x))^2 \right] \quad (1)$$

where $\hat{f}$ is trained on S and (x, y) is a fresh sample from the underlying distribution $\mathcal{D}$. Depending on context, this risk may be conditional on the observed training data (measuring this specific trained model's performance) or averaged over repeated training samples (measuring the training procedure's expected performance).

Connection

Important: Cross-validation estimates the performance of a training procedure. After selecting hyperparameters, we usually refit the final model on all available training data.

Wrong Path: Using Training Error

“Just compute $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2$.”

Why this fails: Training error is **biased downward**—it underestimates test error because $\hat{f}$ was optimized on the same data. For complex models, training error can be zero while test error is large.

Example: Training vs. Test Error

Consider fitting polynomials to $n = 20$ points from $f^*(x) = \sin(2\pi x) + \varepsilon$:

Degree 3:

- Training error: 0.12
- Test error: 0.15 (close!)

Degree 15:

- Training error: 0.01 (excellent!)
- Test error: 0.45 (disaster!)

The degree-15 polynomial memorizes noise, achieving low training error but poor generalization.

Example: Hyperparameter Selection Workflow

Suppose you're tuning ridge regression regularization $\lambda \in \{0.01, 0.1, 1, 10, 100\}$:

The wrong way:

1. Train 5 models on full training set
2. Evaluate on test set
3. Pick best λ based on test performance
4. Report that test performance

Problem: You've used test data for model selection—it's no longer an honest estimate!

The right way:

1. Split training data: 80% train, 20% validation (or use k-fold CV)
2. For each λ : train on train split, evaluate on validation split
3. Pick best λ based on validation performance
4. Retrain on all training data with chosen λ
5. Evaluate once on test set

Result: Test set remains pristine, giving honest performance estimate.

✓ Checkpoint

You have 1000 training samples and use a 70/30 train/validation split to select among 10 candidate models. After selecting the best model, should you (a) report its validation error as the final estimate, or (b) retrain on all 1000 samples and evaluate on a held-out test set? Explain the tradeoff.

Validation Set Approach

The simplest approach: split your training data once, train on one part, validate on the other. Fast but wasteful—you're only training on a fraction of your data.

Definition: Validation Set Method

1. Split data: $S = S_{\text{train}} \cup S_{\text{val}}$ (e.g., 70/30 split)
2. Train on S_{train} : obtain fitted model $\hat{f}_{\text{train}}$
3. Evaluate on S_{val} : compute **validation error** $\hat{R}_{\text{val}} = \frac{1}{|S_{\text{val}}|} \sum_{(x,y) \in S_{\text{val}}} (y - \hat{f}_{\text{train}(x)})^2$

The validation error estimates how well the model will perform on new data.

Advantages:

- Simple and fast
- Unbiased for the test error of the model trained on S_{train}

Disadvantages:

- Wastes data (only trains on subset)
- High variance (depends on random split)
- Validation error typically overestimates the test error of the final model retrained on all data

Pause and Think

Question: If we train on 70% of data, how does the validation error relate to the true test error when training on 100% of data?

Hint: Training on less data typically increases error. Think about learning curves.

Answer:

Validation error is typically **higher** than true test error because $\hat{f}_{\text{train}}$ was trained on fewer samples. The bias depends on the learning curve slope.

→ Week 4 Preview: Learning Curves

Week 4's bias-variance decomposition showed that training on less data increases both bias and variance. Validation set error reflects performance with 30% less training data—often pessimistic for the final model trained on all data.

Progress on backbone: The validation set approach establishes the principle, but it's wasteful—we only train on 70% of our data, and the estimate has high variance from the random split. Can we get the benefits of validation without wasting so much data?

The key insight: instead of a single train/validate split, use **multiple** splits and average the results. k-fold CV partitions data into k pieces, using each piece exactly once for validation while training on the other $k - 1$ pieces. This reuses data efficiently while reducing variance through averaging. The choice of k controls a fundamental tradeoff.

k-Fold Cross-Validation

k-fold CV (k-fold cross-validation) cleverly reuses data: every sample gets to be in the validation set exactly once, and every sample contributes to training. This reduces the data waste problem while controlling variance through averaging.

Definition: k-Fold Cross-Validation

1. Partition S into k disjoint **folds** (equal-sized subsets): $S = S_1 \cup \dots \cup S_k$ with $|S_i| \approx \frac{n}{k}$
2. For $i = 1, \dots, k$:
 - Train on $S \setminus S_i$ (all folds except fold i): obtain model $\hat{f}_{-i}$
 - Validate on S_i (fold i): compute error $\hat{R}_i = \frac{1}{|S_i|} \sum_{(x,y) \in S_i} (y - \hat{f}_{-i}(x))^2$
3. Average over all held-out points: $CV_k = \frac{1}{n} \sum_{i=1}^k \sum_{(x,y) \in S_i} (y - \hat{f}_{-i}(x))^2$

This gives each sample exactly one chance to be in a validation set, while using it for training in all other folds.

Common choices:

- $k = 5$ or $k = 10$: good bias-variance tradeoff
- $k = n$ (LOOCV): nearly unbiased but high variance

Key Takeaway

k-fold CV trades off bias and variance:

- **Small k** (e.g., $k = 5$): trains on less data $\rightarrow$ higher bias, lower variance, faster
- **Large k** (e.g., $k = n$): trains on more data $\rightarrow$ lower bias, higher variance, slower

The optimal k depends on sample size and model complexity.

✓ Checkpoint

Why does large k (like LOOCV with $k = n$) have higher variance than small k (like 5-fold CV)? Think about how much the training sets overlap between folds.

Leave-One-Out Cross-Validation

LOOCV (Leave-One-Out Cross-Validation) is the extreme case: hold out only one sample at a time. Nearly unbiased (trains on $n - 1$ samples, almost the full dataset), but has high variance because training sets overlap heavily—removing one sample barely changes the model, so the n validation errors are highly correlated.

Definition: LOOCV

Special case of k-fold CV with $k = n$. For a loss function $\ell(y, \hat{y})$:

$$CV_n = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{f}_{-i}(x_i)) \quad (2)$$

where $\hat{f}_{-i}$ is trained on all data except the single point (x_i, y_i) .

For regression with squared loss $\ell(y, \hat{y}) = (y - \hat{y})^2$:

$$CV_n = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{f}_{-i}(x_i))^2 \quad (3)$$

Properties:

- Approximately unbiased for the full-data test error when the learning curve changes slowly from $n - 1$ to n
- Deterministic (no random splits)
- High variance (training sets overlap heavily)
- Expensive: requires n model fits

Pause and Think

Question: Why is LOOCV nearly unbiased?

Hint: Each $\hat{f}_{-i}$ is trained on $n - 1$ samples, very close to the full n samples.

Answer:

As $n \rightarrow \infty$, the difference between training on $n - 1$ vs. n samples often vanishes. Thus LOOCV approximates the test error of the full-data training procedure when the learning curve is smooth near n .

Progress on backbone: LOOCV gives nearly unbiased estimates of test error, but at the cost of fitting n models. This seems computationally prohibitive. But here's where we can exploit structure.

For linear models—where predictions are linear in the response vector—there's a remarkable shortcut. We can compute exact LOOCV from a **single** model fit using only the diagonal of the hat matrix. This transforms an $O(n^2 d^2)$ computation into $O(nd^2)$, making LOOCV practical even for large datasets. This is one of the most elegant computational tricks in statistical learning.

Efficient LOOCV for Linear Models

Here's the magic: for linear models (OLS, ridge regression, kernel ridge), we can compute LOOCV without refitting n times! A simple algebraic identity gives exact LOOCV from a single model fit. This makes LOOCV practical even for large n .

The Computational Challenge

Naive LOOCV requires fitting n separate models. For linear regression, each fit costs $O(nd^2)$, giving total cost $O(n^2d^2)$ —prohibitive for large n !

Opening Challenge

Key Question: Can we compute LOOCV without refitting n times?

Answer: Yes! For linear smoothers, there's a closed-form formula requiring only one fit.

Linear Smoothers

Linear smoothers (predictors where predictions are linear combinations of the response values) are models where predictions are linear functions of the response vector $\mathbf{y}$. This includes OLS, ridge regression, and even k-NN! The linearity enables a beautiful algebraic shortcut for LOOCV.

Definition: Linear Smoother

A predictor $\hat{f}$ is a **linear smoother** if predictions are linear in $\mathbf{y}$:

$$\hat{\mathbf{y}} = \mathbf{H}\mathbf{y} \tag{4}$$

where $\mathbf{H} \in \mathbb{R}^{n \times n}$ is the **hat matrix** (also called **smoother matrix** or **influence matrix**; it “puts the hat on $\mathbf{y}$ ” to get predictions $\hat{\mathbf{y}}$). The matrix $\mathbf{H}$ depends on the training inputs $\mathbf{X}$ but not on the response values $\mathbf{y}$.

Example: Linear Smoothers**Ordinary least squares:**

$$\mathbf{H} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$$

Ridge regression:

$$\mathbf{H}_\lambda = \mathbf{X}(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top$$

k-nearest neighbors:

$$H_{ij} = \begin{cases} \frac{1}{k} & \text{if } x_j \text{ is among } k \text{ nearest neighbors of } x_i \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

Why $H_{ii} \neq 1$?

When $H_{ii} = 1$, point i has perfect leverage—the model interpolates it exactly ($\hat{y}_i = y_i$ always). Removing it makes leave-one-out prediction undefined (division by zero).

In practice, this rarely occurs with regularization ($\lambda > 0$ ensures $H_{ii} < 1$). For OLS without regularization, $H_{ii} = 1$ can happen in high-dimensional settings ($d \geq n$).

The LOOCV Shortcut

This is one of the most elegant results in statistical learning: the leave-one-out prediction error can be computed from the full-data fit using only the diagonal elements of the hat matrix. No refitting required!

Theorem: LOOCV for Linear Smoothers

Assume:

- **Fixed design:** Input matrix $\mathbf{X}$ is non-random (we condition on $\mathbf{X}$, only $\mathbf{y}$ varies)
- **Linear smoother:** Fitted values satisfy $\hat{\mathbf{y}} = \mathbf{H}\mathbf{y}$ for some matrix $\mathbf{H}$ (e.g., OLS, ridge regression)
- **Sherman-Morrison identity:** Leave-one-out fit satisfies the rank-1 update formula (holds for OLS, ridge, any linear projection)
- **Leverage constraint:** $H_{ii} \neq 1$ for all i (no perfect leverage points that interpolate exactly)

Then LOOCV can be computed without refitting:

$$\text{CV}_n = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - H_{ii}} \right)^2 \quad (8)$$

where $\hat{y}_i$ is the prediction at x_i using the full data and H_{ii} is the i -th **diagonal element** (the (i, i) entry) of $\mathbf{H}$.

This formula is remarkable: it computes exact LOOCV from a single model fit! The denominator $(1 - H_{ii})$ adjusts the residual $y_i - \hat{y}_i$ to account for the influence of point i on its own prediction.

Proof (Guided Discovery; OLS/ridge deletion sketch):

Step 1: Setup.

Let $\hat{y}_i$ be the prediction at x_i using all n samples, and $\hat{y}_{-i}(x_i)$ be the prediction using all samples except (x_i, y_i) .

We want to relate $\hat{y}_{-i}(x_i)$ to $\hat{y}_i$ without refitting.

Step 2: Sherman-Morrison formula.

The key insight is that removing one sample is a **rank-1 update** (a matrix perturbation that changes the matrix by adding or subtracting an outer product of vectors). For linear regression:

$$\mathbf{X}_{-i}^\top \mathbf{X}_{-i} = \mathbf{X}^\top \mathbf{X} - x_i x_i^\top \quad (9)$$

The **Sherman-Morrison formula** (an identity for efficiently computing the inverse of a rank-1 updated matrix) gives the updated inverse without refitting.

By Sherman-Morrison formula:

$$(\mathbf{A} - uv^\top)^{-1} = \mathbf{A}^{-1} + \frac{\mathbf{A}^{-1}uv^\top \mathbf{A}^{-1}}{1 - v^\top \mathbf{A}^{-1}u} \quad (10)$$

Step 3: Derive the relationship.

For OLS, after algebraic manipulation (details in exercises):

$$\hat{y}_{-i}(x_i) = \hat{y}_i - \frac{H_{ii}}{1 - H_{ii}}(y_i - \hat{y}_i) \quad (11)$$

Rearranging:

$$y_i - \hat{y}_{-i}(x_i) = \frac{y_i - \hat{y}_i}{1 - H_{ii}} \quad (12)$$

Step 4: Compute LOOCV.

$$\text{CV}_n = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_{-i}(x_i))^2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - H_{ii}} \right)^2 \quad (13)$$

Key Takeaway

Computational savings: Instead of $O(n^2 d^2)$ for n refits, we compute:

1. One fit: $O(nd^2)$
2. Diagonal of $\mathbf{H}$: $O(nd^2)$
3. LOOCV formula: $O(n)$

Total: $O(nd^2)$ —same as fitting once! This makes LOOCV practical for large datasets.

Connection

Interpretation of H_{ii} : The diagonal element H_{ii} measures the **leverage** of point i . High leverage means x_i is far from other points, so removing it changes the fit significantly. The denominator $1 - H_{ii}$ inflates the residual accordingly.

✓ **Checkpoint**

What happens to the LOOCV formula when $H_{ii} \rightarrow 1$? Why does it fail for a high-leverage point?

Why GCV $\approx$ LOOCV?

When leverage values H_{ii} have low variance across samples, replacing each with the average $\frac{\text{tr}(\mathbf{H})}{n}$ causes minimal change to the formula.

GCV approximates LOOCV under the assumption that no individual point has disproportionate leverage. This holds for well-conditioned regression problems.

Reference: Craven & Wahba (1979), "Smoothing noisy data with spline functions"

Example: LOOCV Leverage Effect

Consider OLS with $n = 3$ points: $(0, 1)$, $(1, 2)$, $(10, 10)$.

The third point has $x = 10$ far from others. Computing the hat matrix

- $H_{11} \approx 0.5$, $H_{22} \approx 0.5$ (typical leverage)
- $H_{33} \approx 0.98$ (very high leverage!)

If $y_3 - \hat{y}_3 = 0.1$ (small residual):

$$\text{Contribution to } CV_n : \left(\frac{0.1}{1 - 0.98} \right)^2 = \left(\frac{0.1}{0.02} \right)^2 = 25 \quad (14)$$

The high leverage amplifies the residual by 50 $\times$! This makes sense: removing this influential point drastically changes the fit.

Generalized Cross-Validation

GCV replaces individual leverages with their average. This reduces sensitivity to individual high-leverage points while maintaining the computational efficiency of the LOOCV shortcut.

Definition: Generalized Cross-Validation (GCV)

Replace individual leverages H_{ii} with their average:

$$\text{GCV} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i - \hat{y}_i}{1 - \frac{\text{tr}(\mathbf{H})}{n}} \right)^2 \quad (15)$$

where $\frac{\text{tr}(\mathbf{H})}{n}$ is the average leverage (also called effective degrees of freedom divided by n).

Advantages over LOOCV:

- **Rotation invariant:** doesn't depend on coordinate system
- **More stable:** averages out leverage variations
- **Asymptotically equivalent** under mild conditions: $\text{GCV} \approx CV_n$ as $n \rightarrow \infty$ when leverages are not too heterogeneous

Pause and Think

Question: Why is rotation invariance desirable?

Hint: LOOCV can be sensitive to outliers with high leverage. GCV is more robust.

Answer:

Rotation invariance means GCV doesn't depend on arbitrary choices of coordinate axes. This can make it more stable, but if individual leverages vary greatly, LOOCV and GCV can differ substantially.

 Mental Model: Bias-Variance Tradeoff in CV Choice

Think of the CV method choice as navigating a spectrum:

Validation Set (30%) $\leftarrow \rightarrow$ **5-fold CV** $\leftarrow \rightarrow$ **10-fold CV** $\leftarrow \rightarrow$ **LOOCV** $\leftarrow \rightarrow$ **GCV**

Left side (few folds, less data per fit):

- Higher bias (trains on less data)
- Lower variance (less correlation between folds)
- Faster computation
- More stable selection

Right side (many folds, more data per fit):

- Lower bias (trains on nearly all data)
- Higher variance (folds highly correlated)
- Slower computation (but shortcuts exist!)
- More sensitive selection

Mental image: Imagine k overlapping circles representing training sets. Small k = circles barely overlap (independent). Large k = circles almost identical (high correlation $\rightarrow$ high variance).

Rule of thumb:

- Small n (< 100): 5-fold or LOOCV
- Medium n (100-1000): 10-fold
- Large n (> 1000): 5-fold or validation set

✓ **Checkpoint**

You have $n = 50$ samples and two CV strategies: 5-fold vs. LOOCV. For ridge regression, LOOCV uses the closed-form shortcut. Which would you choose and why? Consider both statistical and computational aspects.

 Confidence Check

Rate your understanding (1-5):

- Can you explain why LOOCV has higher variance than 5-fold CV despite being nearly unbiased?
- Can you compute the hat matrix diagonal for simple linear regression?
- Can you decide when to use GCV instead of LOOCV?

If < 4 on any: Review the leverage interpretation and the mental model of fold overlap.

Progress on backbone: Cross-validation gives us empirical estimates of test error by clever reuse of training data. For linear models, we've even eliminated the computational barrier. But cross-validation still requires multiple model fits (or special linear structure). Can we do better?

There's an entirely different approach: **information criteria**. Instead of refitting the model multiple times, these criteria estimate test error from a **single** fit by adding a theoretically-motivated complexity penalty. This connects model selection to fundamental principles from information theory—compression, coding length, and the Kullback-Leibler divergence.

Information-Theoretic Criteria

Cross-validation requires multiple model fits (or clever tricks for linear models). Information criteria offer a different approach: estimate test error from a **single** fit by adding a complexity penalty. This connects model selection to fundamental information theory.

Motivation: Avoiding Expensive CV

Thought Experiment

Suppose we fit a model with k parameters to n samples. The training error is $\hat{R}_{\text{train}}$.

Question: How much should we penalize for model complexity to estimate test error?

Intuition: Each parameter “uses up” some data, reducing effective sample size. The penalty should grow with k and shrink with n .

→ Week 6 Preview: RKHS & Kernel Selection

Week 6 introduces kernel methods where “number of parameters” is ambiguous (potentially infinite-dimensional). The effective degrees of freedom $\text{tr}(\mathbf{H})$ generalizes the parameter count—information criteria still apply with this effective complexity.

Mental Model: Selection Criteria Landscape

Think of model selection criteria as different philosophies about complexity:

Training Error: No penalty

- Philosophy: “Fit the data!”
- Problem: Overfits—ignores generalization

AIC: Penalty = $2k$

- Philosophy: “Predict well on average”
- Target: Expected KL divergence
- Behavior: Prediction-oriented, tolerates slight overfitting

BIC: Penalty = $k \log n$

- Philosophy: “Find the true model”
- Target: Posterior probability
- Behavior: Explanation-oriented, favors parsimony

CV: Data-driven penalty

- Philosophy: “Let the data decide”
- Target: Actual test performance
- Behavior: Empirical, no parametric assumptions

Mental image: Imagine a slider controlling model complexity. Training error says “go right” (more complex). AIC says “stop when prediction plateaus.” BIC says “stop earlier—find the simplest truth.” CV says “let me check empirically.”

When to use each:

- AIC: Prediction focus, model potentially misspecified, medium n
- BIC: True model in candidate set, want sparsity, large n
- CV: No assumptions, willing to pay computational cost

Akaike Information Criterion

AIC emerges from asking: which model minimizes the expected distance (measured by KL divergence) between the true distribution and our fitted model? The answer involves a beautiful bias correction.

Derivation from KL Divergence

Definition: Kullback-Leibler Divergence

The KL divergence from distribution q to p is:

$$\text{KL}(p \parallel q) = \int p(x) \log \left(\frac{p(x)}{q(x)} \right) dx = \mathbb{E}_p[\log p(X)] - \mathbb{E}_p[\log q(X)] \quad (16)$$

Goal: Select model that minimizes KL divergence from true distribution p_{true} to fitted model p_{θ} .

Pause and Think

Question: Can we compute $\text{KL}(p_{\text{true}} \parallel p_{\hat{\theta}})$ directly?

Hint: We don't know p_{true} !

Answer:

No, but we can estimate the **expected** KL divergence over training samples. This leads to AIC.

🏗️ Scaffolded Derivation (AIC from KL Divergence steps)

Goal: Derive the bias correction that leads to AIC.

Step 1: Write expected KL divergence.

The expected KL divergence over training sets S is:

$$\mathbb{E}_S[\text{KL}(p_{\text{true}} \parallel p_{\hat{\theta}})] = \mathbb{E}_S[\mathbb{E}_{x \sim p_{\text{true}}}[\log p_{\text{true}}(x)]] - \mathbb{E}_S[\mathbb{E}_{x \sim p_{\text{true}}}[\log p_{\hat{\theta}}(x)]] \quad (17)$$

Question: Which term depends on our model choice?

Answer: Only the second term (the expected log-likelihood under the true distribution).

Step 2: Focus on the expected log-likelihood.

We want to estimate:

$$\mathbb{E}_S[\mathbb{E}_{x \sim p_{\text{true}}}[\log p_{\hat{\theta}}(x)]] \quad (18)$$

This is the expected log-likelihood on **new** data x drawn from p_{true} , where the expectation is over both the training set S (which determines $\hat{\theta}$) and the new test point x .

Question: What do we have access to?

Answer: The training log-likelihood $\log \mathcal{L}(\hat{\theta}) = \sum_{i=1}^n \log p_{\hat{\theta}}(x_i)$.

Step 3: Identify the bias.

The training log-likelihood is **optimistically biased**:

$$\mathbb{E}[\log \mathcal{L}(\hat{\theta})] > \mathbb{E}_S[\mathbb{E}_{x_{\text{new}}}[\log p_{\hat{\theta}}(x_{\text{new}})]] \quad (19)$$

Why? Because $\hat{\theta}$ was chosen to maximize fit to the training data!

Question: How large is the bias? (This is the hard part!)

Answer: Under regularity conditions, the bias is approximately k , the number of parameters.

Step 4: Derive AIC.

An approximately unbiased estimator of expected log-likelihood on new data is:

$$\log \mathcal{L}(\hat{\theta}) - k \quad (20)$$

Multiplying by -2 (historical convention from likelihood ratio tests):

$$\text{AIC} = -2 \log \mathcal{L}(\hat{\theta}) + 2k \quad (21)$$

Select the model with minimum AIC.

Key insight: Each parameter costs one unit of bias. The penalty $2k$ corrects for overfitting to the training sample.

Theorem: AIC Derivation

Assume:

- **Asymptotic regime:** Fixed model dimension k , sample size $n \rightarrow \infty$
- **Parametric model:** Data follows $p_{\theta(x)}$ with parameter $\theta \in \Theta$
- **Consistent MLE:** $\hat{\theta} \xrightarrow{P} \theta^*$ where θ^* minimizes KL divergence
- **Regularity:** Fisher information matrix positive definite, log-likelihood differentiable

Then the expected log-likelihood on fresh test data satisfies:

$$\mathbb{E}_S[\log p_{\hat{\theta}}(x_{\text{new}})] \approx \log p_{\hat{\theta}}(x_{\text{train}}) - \frac{k}{n} \quad (22)$$

where k is the number of parameters.

Thus, an approximately unbiased estimator of expected log-likelihood is:

$$\log \mathcal{L}(\hat{\theta}) - k \quad (23)$$

Multiplying by -2 gives the **Akaike Information Criterion**:

$$\text{AIC} = -2 \log \mathcal{L}(\hat{\theta}) + 2k \quad (24)$$

Misspecified models

When the true distribution is not in the model family $\{p_{\theta} : \theta \in \Theta\}$, the parameter θ^* is the **KL-best approximation**:

$$\theta^* = \underset{\theta}{\operatorname{argmin}} \operatorname{KL}(p_{\text{true}} \parallel p_{\theta}) \quad (25)$$

AIC still valid—it measures prediction quality of the best approximation, not whether the model is “true.”

Proof sketch: Uses Taylor expansion of log-likelihood around true parameter and properties of maximum likelihood estimators. The bias correction k comes from the trace of the Fisher information matrix.

✓ Checkpoint

Why does the penalty grow with k but not with n ? Hint: Think about what happens to the bias per parameter as n increases.

AIC for Gaussian Linear Models

For regression with Gaussian errors, AIC simplifies to a formula involving the residual sum of squares. This makes it extremely practical for linear models.

Corollary: AIC for Linear Regression

For linear regression with Gaussian errors, up to constants common across models:

$$\text{AIC} = n \log \left(\frac{\text{RSS}}{n} \right) + 2k \quad (26)$$

where $\text{RSS} = \sum_{i=1}^n (y_i - \hat{y}_i)^2$ is the residual sum of squares, and k either includes the variance parameter consistently or it is common to all models being compared.

Proof Template: AIC for Gaussian Models Pattern

Goal: Derive AIC for Gaussian linear regression.

Template: Start from the Gaussian likelihood, substitute the MLE for σ^2 , simplify.

Step 1: Write the log-likelihood.

For $y_i \sim \mathcal{N}(x_i^\top \beta, \sigma^2)$:

$$\log \mathcal{L}(\beta, \sigma^2) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - x_i^\top \beta)^2 \quad (27)$$

Step 2: Substitute MLEs.

The MLE for β minimizes RSS: $\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$

The MLE for σ^2 is: $\hat{\sigma}^2 = \frac{\text{RSS}}{n}$

Step 3: Evaluate log-likelihood at MLEs.

$$\log \mathcal{L}(\hat{\beta}, \hat{\sigma}^2) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log\left(\frac{\text{RSS}}{n}\right) - \frac{n}{2} \quad (28)$$

Step 4: Apply AIC formula.

$$\text{AIC} = -2 \log \mathcal{L} + 2(k+1) \quad (29)$$

$$= n \log(2\pi) + n \log\left(\frac{\text{RSS}}{n}\right) + n + 2k + 2 \quad (30)$$

Step 5: Drop constants.

Constants don't affect model comparison, so:

$$\text{AIC} \approx n \log\left(\frac{\text{RSS}}{n}\right) + 2k \quad (31)$$

(The +1 for σ^2 is often absorbed or counted consistently across models.)

This pattern extends to: Generalized linear models, time series (AICc correction for small n), mixed effects models (with effective degrees of freedom).

Derivation:

For $y_i \sim \mathcal{N}(x_i^\top \beta, \sigma^2)$:

$$\log \mathcal{L}(\hat{\beta}, \hat{\sigma}^2) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log(\hat{\sigma}^2) - \frac{1}{2\hat{\sigma}^2} \text{RSS} \quad (32)$$

With $\hat{\sigma}^2 = \frac{\text{RSS}}{n}$ (MLE):

$$\log \mathcal{L} = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log\left(\frac{\text{RSS}}{n}\right) - \frac{n}{2} \quad (33)$$

Thus:

$$\text{AIC} = -2 \log \mathcal{L} + 2(k+1) \approx n \log\left(\frac{\text{RSS}}{n}\right) + 2k \quad (34)$$

(The $+1$ accounts for σ^2 , often absorbed into the constant.)

Example: AIC for Polynomial Regression

Fit polynomials of degree $d = 1, 2, 3$ to $n = 50$ points:

Degree 1: $\text{RSS}_1 = 45$, $k = 2$ (slope + intercept)

$$\text{AIC}_1 = 50 \log\left(\frac{45}{50}\right) + 2(2) = 50 \log(0.9) + 4 \approx -5.3 + 4 = -1.3 \quad (35)$$

Degree 2: $\text{RSS}_2 = 30$, $k = 3$

$$\text{AIC}_2 = 50 \log\left(\frac{30}{50}\right) + 2(3) = 50 \log(0.6) + 6 \approx -25.5 + 6 = -19.5 \quad (36)$$

Degree 3: $\text{RSS}_3 = 28$, $k = 4$

$$\text{AIC}_3 = 50 \log\left(\frac{28}{50}\right) + 2(4) = 50 \log(0.56) + 8 \approx -28.9 + 8 = -20.9 \quad (37)$$

Selection: Choose $d = 3$ (lowest AIC). The RSS improves slightly from degree 2 to 3, and the penalty only increases by 2, so the more complex model wins.

Properties of AIC

Key Takeaway

AIC properties:

1. **Often asymptotically related to LOOCV** for regular linear models
2. **Not generally consistent:** it may not select the smallest true model as $n \rightarrow \infty$
3. **Prediction-oriented:** estimates expected out-of-sample KL risk
4. **Can overfit:** penalty $2k$ is relatively small
5. **Useful under misspecification:** targets the best KL approximation in the candidate family

Example: AIC Asymptotic Equivalence to LOOCV

For linear regression, as $n \rightarrow \infty$:

$$\text{AIC} \approx n \log\left(\frac{\text{RSS}}{n}\right) + 2k \quad (38)$$

$$\text{CV}_n \approx n \log\left(\frac{\text{RSS}}{n}\right) + 2k \quad (39)$$

The equivalence holds because both estimate the same quantity: expected prediction error.

← From Week 2: VC Dimension & Generalization

Week 2's VC bounds gave worst-case complexity penalties $O\left(\sqrt{\frac{d}{n}}\right)$. AIC's penalty $2\frac{k}{n}$ is much smaller—it's an average-case analysis under correct parametric specification, not a worst-case bound. This is why AIC can underpenalize when models are severely misspecified.

Bayesian Information Criterion (BIC)

Motivation: AIC minimizes prediction error on future data. But what if our goal is to identify the **true model** (assuming it exists in our candidate set)? The Bayesian Information Criterion (BIC) takes a different approach: select the model with highest posterior probability given the data.

Theorem: BIC from Bayesian Posterior

Under regularity conditions (consistent MLE, bounded prior density, fixed model dimension k), the log posterior probability of model $\mathcal{M}_j$ given data S satisfies:

$$\log P(\mathcal{M}_j | S) \approx \log L(\hat{\theta}_j; S) - \frac{k_j \log n}{2} + O(1) \quad (40)$$

where $L(\hat{\theta}_j; S)$ is the likelihood at the MLE $\hat{\theta}_j$ and k_j is the parameter count.

Maximizing the posterior is equivalent to minimizing:

$$\text{BIC} = -2 \log L(\hat{\theta}) + k \log n \quad (41)$$

Proof. By Bayes' theorem: $P(\mathcal{M} | S) \propto P(S | \mathcal{M})P(\mathcal{M})$.

The marginal likelihood $P(S | \mathcal{M}) = \int P(S | \theta, \mathcal{M})P(\theta | \mathcal{M})d\theta$ integrates over parameters. Using Laplace approximation around MLE $\hat{\theta}$:

$$\log P(S | \mathcal{M}) \approx \log L(\hat{\theta}) - \frac{k}{2} \log n + O(1) \quad (42)$$

The $k \log n$ term comes from the Gaussian approximation to the posterior (Fisher information has dimension $k \times k$). Taking negative log and dropping constants gives BIC. ■

Theorem: BIC Consistency

If the true model $\mathcal{M}^*$ is in the candidate set $\{\mathcal{M}_1, \dots, \mathcal{M}_K\}$ and $n \rightarrow \infty$:

$$\mathbb{P}(\text{BIC selects } \mathcal{M}^*) \rightarrow 1 \quad (43)$$

By contrast, AIC is **not consistent**—it asymptotically selects models that minimize prediction error, which may include parameters with negligible effects.

Proof. Sketch: For the true model, $-2 \log L(\hat{\theta}^*) = O_{P(1)}$, so $\text{BIC}^* = O_{P(1)} + k^* \log n$.

For overfitted models containing $\mathcal{M}^*$, the log-likelihood improves by $O_{P(1)}$ but the penalty increases by $(k - k^*) \log n \rightarrow \infty$. Thus overfitted models have $\text{BIC} \rightarrow \infty$ in probability.

For underfitted models missing $\mathcal{M}^*$, the KL divergence causes $-2 \log L(\hat{\theta}) = O_{P(n)}$, dominating the $k \log n$ penalty. BIC correctly rejects these. ■

Key Takeaway**AIC vs BIC:**

- **AIC** ($2k$ penalty): Minimizes prediction error, not consistent, better for small n
- **BIC** ($k \log n$ penalty): Selects true model (if in candidate set), consistent as $n \rightarrow \infty$, more conservative

Example: AIC vs BIC in Practice

Fit polynomial models of degree $d = 1, 2, 3, 4$ to $n = 100$ data points. Suppose true model is degree 2.

- **AIC**: Might select $d = 3$ or $d = 4$ because penalty $2k$ is small, allowing slight overfitting for better short-term prediction
- **BIC**: More likely selects $d = 2$ (true model) because penalty $k \log(100) \approx 4.6k$ strongly penalizes extra parameters

For $n = 10,000$:

- **AIC**: Still selects $d = 3$ or $d = 4$
- **BIC**: Almost certainly selects $d = 2$ because penalty $k \log(10000) \approx 9.2k$ is very large

Comparison of Model Selection Criteria

Criterion	Penalty	Consistent?	Use when...
AIC	$2k$	No	Prediction tasks, model averaging, small samples
BIC	$k \log n$	Yes	Scientific inference, want parsimonious model, large n
LOOCV	Data-driven	Depends	Computational budget allows, model-agnostic

Practical Considerations

Real-world model selection involves more than choosing a criterion. You need to avoid selection bias, manage computational costs, and understand the limitations of each approach.

Nested Cross-Validation

One of the most common mistakes in machine learning: using the same data for both model selection and performance estimation. This creates selection bias.

Wrong Path: Selection Bias

“Use k-fold CV to select λ , then report the CV error as test error estimate.”

Why this fails: The CV error is optimistically biased because λ was chosen to minimize it. This is **selection bias**.

Definition: Nested Cross-Validation

Outer loop (performance estimation):

- Split data into k_{out} folds
- For each outer fold i :

Inner loop (hyperparameter selection):

- Split training data into k_{in} folds
- Use inner CV to select λ_i^*
- Train on full outer training set with λ_i^*
- Evaluate on outer test fold i

Report: Average outer fold errors (a less biased estimate of performance after the full selection procedure)

Key Takeaway

Nested CV separates:

1. **Model selection** (inner loop): choose hyperparameters
2. **Performance estimation** (outer loop): estimate test error

This avoids selection bias at the cost of more computation.

Example: Selection Bias in Action

Suppose we try $\lambda \in \{0.01, 0.1, 1, 10, 100\}$ for ridge regression:

Wrong approach: 5-fold CV to select best λ

- Best $\lambda = 1$ with CV error = 0.20
- Report: “Model achieves error 0.20”

Problem: We tried 5 values, picked the best. The 0.20 is optimistic!

Right approach: Nested CV (5 outer $\times$ 5 inner)

- Inner CV selects λ for each outer fold
- Outer CV evaluates the entire selection procedure
- Report: “Model selection procedure achieves error 0.23”

The honest estimate is higher (0.23 vs 0.20) because it accounts for selection.

🔥 Productive Struggle

Challenge: You have 500 samples and want to tune two hyperparameters (λ and kernel bandwidth γ) using a grid of $10 \times 10 = 100$ combinations. Design a nested CV scheme.

Questions to resolve:

1. How many folds for outer and inner loops?
2. How many total model fits?
3. What’s the computational cost compared to simple validation set?
4. Is there a way to reduce cost without sacrificing validity?

Extension: What if you have 50 hyperparameter combinations? At what point does nested CV become impractical?

Computational Complexity Comparison**Example: Complexity Summary**

For dense linear models with $n \geq d$ and closed-form least-squares fits:

Method	Cost	Notes
LOOCV (naive)	$O(n^2 d^2)$	Refit n times
LOOCV (closed form)	$O(nd^2)$	One fit + diagonal
k -fold CV	$O(knd^2)$	k refits
GCV	$O(nd^2)$	One fit + trace
AIC/BIC	$O(nd^2)$	One fit + likelihood

For nonlinear models: No shortcuts—must refit for CV.

✓ Checkpoint

You have $n = 10,000$ samples, $d = 100$ features, and a nonlinear SVM. Comparing 5-fold CV, 10-fold CV, and LOOCV for hyperparameter selection over 50 candidates, estimate the relative computational costs. Which would you choose?

Common Pitfalls

Opening Challenge

Pitfall 1: Using test data for selection Never use test data to choose models. Test set is only for final evaluation.

Pitfall 2: Ignoring selection uncertainty After selecting a model, confidence intervals should account for selection. Standard intervals are too narrow.

Pitfall 3: Overfitting to validation set If you try many models, you can overfit to the validation set. Use nested CV or hold out a final test set.

Pitfall 4: Wrong penalty for small n AIC/BIC are asymptotic. For small n , use corrected versions (AICc) or CV.

🔧 Construct Your Own

Design a model selection workflow for a real problem:

Scenario: Image classification with 2000 training images. You're comparing:

- Logistic regression with polynomial features (degree 1-5)
- SVM with RBF kernel (3 bandwidth values $\times$ 3 C values)
- Random forest (3 tree counts $\times$ 3 max depth values)

Design choices:

1. Single validation set, k-fold CV, or nested CV?
2. What k for CV?
3. How to avoid overfitting to validation set?
4. How to report final performance honestly?
5. Computational budget: assume each model takes 1 minute to train.

Calculate: Total training time for your workflow.

Justify: Why is your workflow appropriate for this scenario?

Connection to Learning Theory: Structural Risk Minimization

Motivation: AIC and BIC provide model selection under parametric assumptions (correct likelihood, asymptotic regime). But what if we want distribution-free guarantees? Structural Risk Minimization (SRM) from statistical learning theory provides worst-case model selection using VC theory.

Definition: Structural Risk Minimization (SRM)

Given a nested sequence of hypothesis classes:

$$\mathcal{H}_1 \subset \mathcal{H}_2 \subset \dots \subset \mathcal{H}_K \quad (44)$$

with increasing VC dimensions $d_1 \leq d_2 \leq \dots \leq d_K$, SRM selects the hypothesis that minimizes:

$$\hat{h}_{\text{SRM}} = \underset{h \in \bigcup_{k=1}^K \mathcal{H}_k}{\operatorname{argmin}} \left[\hat{R}_S(h) + \text{penalty}(d_k, n, \delta) \right] \quad (45)$$

where the penalty ensures uniform convergence via VC bounds from Week 2.

← From Week 2: VC Dimension & Fundamental Theorem

Week 2 showed that with probability $\geq 1 - \delta$:

$$R_{\mathcal{D}}(h) \leq \hat{R}_S(h) + O\left(\sqrt{\frac{d \log\left(\frac{n}{d}\right) + \log\left(\frac{1}{\delta}\right)}{n}}\right) \quad (46)$$

SRM uses this bound as the penalty term, balancing empirical risk (fit to data) against model complexity (VC dimension).

Theorem: SRM Generalization Bound

With probability $\geq 1 - \delta$ over training samples $S \sim \mathcal{D}^n$:

$$R_{\mathcal{D}}(\hat{h}_{\text{SRM}}) \leq \min_{k=1, \dots, K} \left[\inf_{h \in \mathcal{H}_k} R_{\mathcal{D}}(h) + \varepsilon_{k(n, \delta)} \right] \quad (47)$$

where $\varepsilon_{k(n, \delta)} = O\left(\sqrt{\frac{d_k \log n + \log K}{n}}\right)$ is the VC convergence rate for class $\mathcal{H}_k$.

Key Takeaway

SRM provides a **distribution-free** model selection principle:

- No parametric assumptions (unlike AIC/BIC)
- Worst-case guarantees (unlike cross-validation's empirical estimates)
- Explicit complexity-accuracy tradeoff via VC dimension

When to use SRM?

- Non-parametric settings (no likelihood model)
- Need worst-case guarantees
- Model classes with known VC dimensions

When to use AIC/BIC?

- Parametric models (regression, classification)
- Computational efficiency (one fit per model)
- Interpretation (likelihood, posterior probability)

Example: SRM for Polynomial Regression

Consider nested classes $\mathcal{H}_d = \{\text{degree-}d \text{ polynomials}\}$ for $d = 1, 2, 3, 4, 5$.

VC dimensions: $\text{VC}(\mathcal{H}_d) = d + 1$ (Week 2).

For $n = 100$ samples and $\delta = 0.05$, SRM computes:

$$\text{Score}_d = \hat{R}_S(\hat{h}_d) + \sqrt{\frac{d \log(100) + \log(5)}{100}} \quad (48)$$

- $d = 1$: $\hat{R}_S(\hat{h}_1) = 0.40$, penalty ≈ 0.08 , score = 0.48
- $d = 3$: $\hat{R}_S(\hat{h}_3) = 0.15$, penalty ≈ 0.14 , score = 0.29
- $d = 5$: $\hat{R}_S(\hat{h}_5) = 0.10$, penalty ≈ 0.18 , score = 0.28 ← **selected**

SRM balances the reduction in training error against increasing complexity.

AIC vs BIC vs SRM

Criterion	Penalty	Assumptions	Consistent?	Guarantee
AIC	$2k$	Parametric	No	Average-case prediction
BIC	$k \log n$	Parametric + true model	Yes	Selects true model
SRM	$O\left(\sqrt{\frac{d}{n}}\right)$	None (distribution-free)	Yes	PAC bound on test error
LOOCV	Data-driven	None	Varies	Empirical estimate

→ Week 6 Preview: Kernel Methods

Week 6 introduces kernel ridge regression with infinite-dimensional feature spaces. SRM extends naturally: the VC dimension is replaced by the trace of the kernel matrix (effective dimensionality), and the same complexity-accuracy tradeoff applies.

Summary and Connections

Key Insights

1. **Cross-validation** estimates test error by simulating train/test splits
2. **LOOCV** is nearly unbiased but has efficient closed forms for linear models
3. **AIC** minimizes prediction error; derived from KL divergence
4. **BIC** selects true model consistently; derived from Bayesian principles
5. **MDL** formalizes Occam's razor via coding theory
6. **SRM** provides distribution-free guarantees via VC theory

Connection

To Week 4: Regularization parameter λ is selected via CV or information criteria.

To Week 6: Kernel selection in RKHS uses same principles—CV for hyperparameters, complexity penalties for model selection.

To Week 9-10: Boosting iterations as model complexity—when to stop? Use CV or validation set.

To Week 11-12: Neural architecture search and early stopping—modern applications of model selection.

Synthesis Challenge

The Model Selection Toolbox:

You've now learned five approaches to model selection. For each scenario below, recommend the best approach and justify your choice:

Scenario A: $n = 50$ samples, comparing polynomial degrees 1-5, linear regression.

- Options: Validation set, 5-fold CV, LOOCV, AIC, BIC

Scenario B: $n = 10,000$ samples, comparing 100 neural network architectures, nonlinear.

- Options: Validation set, 5-fold CV, 10-fold CV, early stopping

Scenario C: $n = 200$ samples, comparing 5 linear models, need to find true sparse features.

- Options: 10-fold CV, LOOCV, AIC, BIC, SRM

Scenario D: $n = 1,000$ samples, ridge regression with 50 candidate λ values.

- Options: LOOCV with shortcut, 5-fold CV, AIC, nested CV

For each:

1. What's your recommendation?
2. What are the tradeoffs?
3. What assumptions are you making?
4. How would you report the final performance?

Closing Challenge

Opening Challenge

Revisiting the Opening Question:

For ridge regression with $n = 100$, $d = 50$:

1. Validation set (70/30):

- Pros: Fast, simple
- Cons: Wastes 30 samples, high variance
- Use when: n is large, speed matters

2. LOOCV:

- Pros: Nearly unbiased, uses all data
- Cons: High variance, expensive (but $O(nd^2)$ with shortcut!)
- Use when: n is small, accuracy matters

3. AIC:

- Pros: Fastest ($O(nd^2)$), no refitting
- Cons: Assumes model family, asymptotic
- Use when: Model is well-specified, n is large

Recommendation: For $n = 100$, use LOOCV with closed form (fast and accurate) or 10-fold CV (good tradeoff). AIC is reasonable if Gaussian assumptions hold.

Confidence Check

Rate your mastery of Week 5 (1-5):

Cross-Validation:

- Can you explain the bias-variance tradeoff in k-fold CV?
- Can you derive the LOOCV shortcut for linear models?
- Can you implement nested CV to avoid selection bias?

Information Criteria:

- Can you derive AIC from KL divergence?
- Can you explain when BIC is consistent and when AIC is not?
- Can you compute AIC/BIC for Gaussian linear models?

Practical Skills:

- Can you choose the right selection method for a given scenario?
- Can you avoid common pitfalls like selection bias?
- Can you estimate computational costs for different approaches?

If < 4 on any: Review the corresponding sections, work through examples, and try the construct-your-own exercises.

Student Challenge

Exercise: Implement efficient LOOCV for ridge regression. Compare with naive refitting and verify the speedup. Then compare LOOCV, 10-fold CV, and AIC for selecting λ on a real dataset.

Research question: Modern deep learning uses millions of parameters but doesn't use traditional model selection criteria. Why do AIC/BIC fail for neural networks? What do we use instead? (Preview: Week 11 discusses implicit regularization and early stopping.)